

📖 Langchain Advanced Agents — Concise Notes

1. Recap — Basic Agent Loop

Standard agent execution (previously covered): **Request** → **Model** → (**Action** ↔ **Observation** ↔ **Tools**) → **Result**. The agent reasons about a request, uses tools (Action), observes results, and returns a Result.

2. Middleware — Dynamic Resource Selection

The **Middleware API in Langchain** allows the developer to **control the internal workings of the agent** — inserting ε ($\times$) hooks at key points (before the model, between Action ↔ Observation, and before the Result).

Middleware is useful for:

- **Tracking** agent behavior with logging, analytics, and debugging.
- **Transforming** prompts, tool selection, and output formatting.
- Adding **retries, fallbacks, and early termination** logic.
- Applying **rate limits, guardrails, and PII detection**.

Guardrails help build safe, compliant AI applications by **validating and filtering content at key points** in the agent's execution. They can detect sensitive information, enforce content policies, validate outputs, and prevent unsafe behaviors before they cause problems.

3. Why Standard Request/Response Isn't Feasible with LLMs

Long conversations (A, B, C, D, E, F, G, H...) sent straight to an LLM → **slow (turtle)**, **declining performance**, and **high cost (\$)**. We need smarter ways to manage long chats.

4. Managing Long Conversations — Two Strategies

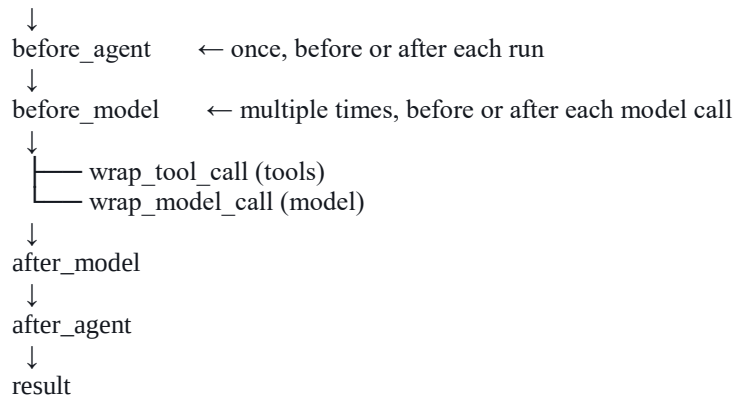
As conversation grows (A, B, C, D → A, B, C, D, E, F), we need to manage context. Two basic techniques:

1. **Trim Conversation** → Compress old messages into a single summary (e.g., A, B, C, D, E, F → "ABCDEF..." + G!).
2. **Deleting Messages** → Drop old messages, keeping only the most recent (e.g., keep only E, F, G).

5. Middleware Hooks — Execution Flow

Middleware exposes hooks **before and after** each step inside the agent:

request



Prebuilt Middleware (Langchain ships many; three highlighted):

Middleware	Description
Summarization	Automatically summarize conversation history when approaching token limits.
Human-in-the-loop	Pause execution for human approval of tool calls.
File search	Provide Glob and Grep search tools over filesystem files.

6. Managing Long Chats — Code

python

```

from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware

agent = create_agent(
    model="gpt-4.1",
    tools=[...],
    middleware=[
        SummarizationMiddleware(...),
        HumanInTheLoopMiddleware(...)
    ],
)

```

7. Summarization Middleware — Full Example

python

```

from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import SummarizationMiddleware

llm = create_azure_llm()

agent = create_agent(
    model=llm,

```

```

checkpointer=InMemorySaver(),
middleware=[
    SummarizationMiddleware(
        model=llm,
        trigger=("tokens", 100), # summarize when token count exceeds 100
        keep=("messages", 1)    # keep the last 1 message after summarization
    )
]
)

```

Key parameters:

- `model` — the LLM used to produce the summary.
- `trigger=("tokens", 100)` — condition that triggers summarization.
- `keep=("messages", 1)` — how much recent history to retain after summarization.

8. Controlling the ReAct Loop Through Middleware

Two categories of middleware hooks, based on **when & how often** they fire:

Decorator	When it runs
@before_agent f(x) / @after_agent f(x)	Once , before or after each run (the whole agent invocation).
@before_model f(x) / @after_model f(x)	Multiple times , before or after each model call (every loop iteration).

9. Human-in-the-Loop (HITL)

Useful for:

- **High-stakes operations** requiring human approval (e.g., database writes, financial transactions).
- **Very common in VSCode Agent-based tasks in Copilot chat.**
- **Compliance workflows** where human oversight is mandatory.
- **Long-running conversations** where human feedback guides the agent.

Human-In-the-Loop Middleware pauses agent execution for human **approval, editing, or rejection** of tool calls before they execute.

How it works:

- When a model proposes an action that might require review (e.g., writing to a file, executing SQL), the middleware can **pause execution** and wait for a decision.
- It does this by checking each tool call against a **configurable policy**.
- If intervention is needed, the middleware issues an **interrupt** that halts execution.
- The graph state is saved using **LangGraph's persistence layer**, so execution can pause safely and resume later.
- A human decision determines what happens next.

10. Interrupt Decision Types

Decision	Description	Example Use Case
✓ approve	Action approved as-is, executed without changes.	Send an email draft exactly as written.
✎ edit	Tool call executed with modifications .	Change the recipient before sending an email.
✗ reject	Tool call rejected, with explanation added to the conversation.	Reject an email draft and explain how to rewrite it.

The available decision types for each tool **depend on the policy you configure in `interrupt_on`**.

11. Human-in-the-Loop Policies — Code

python

```
agent = create_agent(
    model="gpt-4.1",
    tools=[write_file_tool, execute_sql_tool, read_data_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                # All decisions (approve, edit, reject) allowed
                "write_file": True,
                # No editing allowed
                "execute_sql": {"allowed_decisions": ["approve", "reject"]},
                # Safe operation, no approval needed
                "read_data": False,
            },
            description_prefix="Tool execution pending approval",
        ),
    ],
    # In production, use a persistent checkpointer like AsyncPostgresSaver.
    checkpointer=InMemorySaver(),
)
```

Policy interpretation:

- `True` → all decisions (approve, edit, reject) allowed.
- `{"allowed_decisions": [...]}` → restrict to specific decisions.
- `False` → no approval needed (safe operation).

12. Dynamic Prompts — `@dynamic_prompt` Decorator

- `@dynamic_prompt` is an **LCEL (LangChain Expression Language)** feature that lets you define prompts as **Python functions** instead of manually constructing `PromptTemplate` or `ChatPromptTemplate`.
- Creates **runtime-adaptive prompts** whose structure, messages, or content can change based on logic inside the function.

- `@dynamic_prompt` turns a **Python function into a prompt-producing runnable**:
 - **Inputs** → function arguments
 - **Output** → prompt messages (or text)
 - **Logic** → fully dynamic

13. Dynamic Agents

Real-world scenarios require **dynamic flows** — different users/roles/situations require different prompts, tools, and models. Static agents don't scale; we need agents that adapt at runtime.

14. Node-Style Middleware (so far)

The "pipeline of nodes/runnables" is **node-style middleware** (*node* here means a step in the pipeline, analogous to Node.js-style middleware chains — **not** Node.js itself).

- Middleware sits **between steps** (between User ↔ Model ↔ Tools inside the Agent).
- Can **modify input or output**.
- Can **log, validate, retry, timeout**, etc.

15. Model Request

A **Model Request** is made up of three components fed into the Model:

- **Prompt**
- **Tools**
- **State**

16. Dynamic Agents — Intercepting the Model Request

ModelRequest is a data structure within Langchain Middleware to **intercept, inspect, or modify the request before it reaches the LLM**.

Fields of `ModelRequest`:

Field	Description
<code>system_message / system_prompt</code>	The current system instructions for the model.
<code>messages</code>	The full list of conversation messages being sent.
<code>tools</code>	The list of tools available to the model for this specific call.
<code>model_settings</code>	Dictionary for runtime overrides (e.g., <code>temperature</code> , <code>max_tokens</code> , or provider-specific headers).
<code>state</code>	The current agent state — allows modifying the request based on previous-turn data.
<code>runtime</code>	Access to the execution context, including metadata and the Store (for persistent memory).

17. @wrap_model_call

A **core utility** designed to provide a **consistent wrapper around the execution of LLMs**. Acts as a **decorator** that handles the **lifecycle** of a model interaction:

1. **Preprocessing** — prepares input prompts and checks for any active **Callbacks** (e.g., logging).
2. **Dispatching** — triggers the `on_llm_start` event.
3. **Execution** — executes the actual model call (the "wrapped" function).
4. **Post-processing**:
 - **On Success** — captures the `LLMResult`, triggers `on_llm_end`, returns the output.
 - **On Error** — captures the exception, triggers `on_llm_error`, and **re-raises the error** so your code can handle it.

18. Design Guidance — Which Middleware Style to Use?

	Wrap-style Middleware	Node-style Middleware	Tool Calls
Operates on	<code>ModelRequest</code>	<code>State + Runtime</code>	<code>ToolRuntime</code>
Purpose	Adjust an agent's tools, prompt, model while running	Adjust an agent's state while running	Agent adjusts its own state or retrieves runtime context
Example	Giving an agent access to tools if the user is internal	Trimming messages from long conversations	Updating the state with the username when provided to the agent

Quiz Questions

◆ B. Technical / Code-Based

Q27. Complete the code to add Summarization and HITL middleware:

```
python
from langchain.agents import create_agent
from langchain.agents.middleware import import _____, _____

agent = create_agent(
    model="gpt-4.1",
    tools=[...],
    middleware=[_____ (...), _____ (...)],
```

)

Answer: SummarizationMiddleware, HumanInTheLoopMiddleware.

Q28. In the SummarizationMiddleware example, what do `trigger=("tokens", 100)` and `keep=("messages", 1)` mean? **Answer:** Summarization is **triggered** when the conversation reaches **100 tokens**; after summarizing, the middleware **keeps the last 1 message** of recent history.

Q29. Given this HITL policy:

```
python
interrupt_on={
    "write_file": True,
    "execute_sql": {"allowed_decisions": ["approve", "reject"]},
    "read_data": False,
}
```

What decisions are allowed for each tool? **Answer:**

- `write_file` → **all** decisions allowed (approve, edit, reject).
- `execute_sql` → only **approve** and **reject (no edit)**.
- `read_data` → **no approval needed** (safe operation, runs directly).

Q30. Which checkpointer is recommended for production HITL (per the slide), and which is used in the demo code? **Answer:** Production → `AsyncPostgresSaver` (persistent). Demo → `InMemorySaver`.

Q31. Which import provides `InMemorySaver` in the Summarization Middleware example?

Answer: `from langgraph.checkpoint.memory import InMemorySaver`.

Q32. Which decorators run **multiple times per run** during the ReAct loop? **Answer:**

`@before_model` and `@after_model`.

Q33. In `ModelRequest`, which field would you modify to change the list of tools available for a specific call? **Answer:** The `tools` field.

Q34. Which field of `ModelRequest` would you use to override `temperature` at runtime?

Answer: `model_settings`.

◆ C. Output-Based / Scenario

Q35. A user is making a DB write via an agent. You want a human to approve before it's executed. Which middleware and which decision types apply? **Answer:** Use **HumanInTheLoopMiddleware**. The user can decide to **approve, edit, or reject** the tool call (subject to the `interrupt_on` policy).

Q36. Your agent's conversation history has grown past the token limit. Which middleware fixes this automatically, and what key parameters would you set? **Answer:**

SummarizationMiddleware, with `trigger=("tokens", <limit>)` and `keep=("messages", <N>)`.

Q37. You want to decide at runtime whether to give an agent extra tools based on whether the user is internal or external. Which middleware style fits best and why? **Answer:** **Wrap-style Middleware** (operates on `ModelRequest`) — purpose is to adjust an agent's tools/prompt/model while running. The example in Design Guidance matches this exactly.

Q38. When the user's request first arrives, what is the very first middleware hook that fires?

Answer: `before_agent` (once per run, before `before_model`).

Q39. An error happens during the model call. What does `@wrap_model_call` do? **Answer:** It captures the exception, triggers `on_llm_error`, and **re-raises the error** so your code can handle it.

Q40. Why is `@dynamic_prompt` better than `PromptTemplate / ChatPromptTemplate` when prompts must depend on runtime logic (e.g., user role, A/B tests)? **Answer:** Because it turns a Python function into a prompt-producing runnable — inputs become function arguments, output is prompt messages, and the logic is fully dynamic, so structure/messages/content can change based on logic inside the function.

Q41. A successful LLM call through `@wrap_model_call` produces what, and which event is triggered? **Answer:** It captures an `LLMResult` and triggers the `on_llm_end` event before returning the output.

◆ D. Quiz-POV Rapid Fire

- **What does `trigger=("tokens", 100)` do?** → Summarize when conversation exceeds 100 tokens.
- **What does `keep=("messages", 1)` do?** → Keep the last 1 message after summarization.
- **True/False: `@before_agent` runs multiple times per run.** → False (it runs once).
- **True/False: `@before_model` runs multiple times per run.** → True (once per model call).
- **Which decision is used when a tool call should proceed but with changes?** → **edit**.
- **Which decision adds feedback/explanation to the conversation?** → **reject**.
- **A tool configured with `False` in `interrupt_on` means?** → No approval needed — safe operation runs directly.
- **A tool configured with `True` in `interrupt_on` allows?** → All three decisions (approve, edit, reject).
- **`ModelRequest` is used to intercept the request before it reaches the _____. → LLM.**
- **Which middleware is "very common in VSCode Agent-based tasks in Copilot chat"?** → Human-in-the-Loop.
- **Node-style middleware operates on?** → State + Runtime.

- **Tool Calls operate on?** → `ToolRuntime`.
- **File search middleware uses which two tools?** → **Glob** and **Grep**.
- **What does `@dynamic_prompt` belong to (feature family)?** → LCEL (LangChain Expression Language).
- **Three stages of Model Request inputs?** → Prompt, Tools, State.
- **LangGraph's _____ layer lets HITL pause safely and resume later.** → persistence.